Explainora: A Closed-Loop, Token-Efficient Pipeline for Narrated Educational Video Generation

Author names, affiliations, and correspondence: to be supplied by the authors

Manuscript draft, September 2026. Author metadata and controlled-study results require researcher input before submission.

Abstract

Automatically turning a short question into a pedagogically coherent video requires coordinated narration, visual planning, animation, speech synthesis, and quality control. Existing code-generating and agentic approaches demonstrate the value of multimodal explanations but can incur substantial language-model token usage, repeated repair calls, layout defects, and unpredictable narration duration. This paper presents **Explainora**, a CPU-rendered pipeline that converts prompts into narrated, animated vector scenes. Its implemented *cross-job closed loop* uses measured raw speech duration from a voice profile to update future narration word budgets; within a job, duration-word, factuality, visual-layout, and continuity checks run before finished-file publication. A compact scene representation and locally executed animation, continuity, layout, and quality operations move substantial work out of the language model. The system records provider-reported prompt, completion, cached, and reasoning tokens when available. In a two-topic local comparison of its quality settings, all four 120-second attempts produced playable MP4s. The fast setting's median total was 55,949 tokens versus 49,600.5 for balanced (**12.8% more**), while its model-list-price estimate was US$0.01557 versus US$0.07646 (**79.6% less**). Sampled scenes contain visual defects and no blinded human ratings exist. Thus cheaper provider routing is observed, but fewer tokens per acceptable video or superiority over another system is **not** established. A larger, frozen, matched-baseline study is needed.

Index Terms - Educational video generation, large language models, vector graphics, closed-loop control, text-to-speech, token efficiency, multimodal explanation.

I. INTRODUCTION

Educational videos are useful when they reveal a process at the moment it is explained, rather than merely placing text beside images. Creating such material manually demands writing, illustration, animation, voice recording, editing, and timing. Recent systems automate portions of this workflow with LLM-generated Manim code or multimodal agents [1]-[4]. TheoremExplainAgent, for example, generates long-form theorem videos, but its authors report recurring layout problems and large token consumption when retrieval and code repair are involved [1]. VisualEDU likewise identifies logical, temporal, and visual clarity as separate failure modes in educational problem-solving videos [3]. More recent work examines structured whiteboard strokes aligned to speech [5] and interleaved text-diagram explanations [6].

Explainora addresses a related but distinct task: generating a narrated explainer from an open-ended question with an audience, a visual style, and a target duration. The interface also accepts PDF, Word, presentation, Markdown, and text uploads; however, the current production batch worker does **not yet pass the extracted text to the director**. Uploaded-document grounding is therefore an implementation gap, not an evaluated capability. Instead of asking an LLM to write a complete animation program or generating raster frames with a diffusion model, Explainora asks models for a bounded educational script and structured scene elements. SVG, narration audio, captions, and MP4 segments are assembled with local software.

The paper's research question is: **Can a closed-loop, symbolic pipeline deliver pedagogically useful video while spending fewer language-model tokens per *successful* output than code-heavy or repeatedly regenerated alternatives?** This is a token claim, not a blanket claim that Explainora has lower total compute or lower cost than every commercial video product. Commercial platforms often do not publish token counts, may use non-LLM rendering costs, and serve different video durations and styles. The defensible comparison is controlled, matched-task evaluation, supplemented by carefully qualified published figures.

Our contributions are:

1. A voice-profile-keyed feedback controller that uses measured raw TTS duration to update narration budgets for **subsequent** jobs, with bounded **pre-TTS word-contract** repair within a job.

2. A structured vector-scene representation that combines visual elements and compact animation directives, followed by deterministic narration alignment and rendering without generating full animation source code.

3. Pre-TTS duration and factuality checks plus local visual-layout and continuity operations that preserve accepted scene plans where possible, instead of defaulting to full video regeneration.

4. A per-call usage ledger and an evaluation protocol that reports prompt/output/reasoning tokens, retry overhead, cost, latency, success rate, and instructional quality jointly.

II. RELATED WORK

A. Educational video generation

TheoremExplainAgent uses agentic planning and Manim to generate videos explaining STEM theorems and introduces a 240-theorem benchmark [1]. Its published cost and token table is an important systems baseline, but its videos are typically longer and more mathematically specialized than Explainora's open-domain explainers. Manimator converts research papers or prompts into scene descriptions and then Manim code [2]. PresentAgent transforms long documents into narrated presentation videos through document segmentation, visual planning, TTS, and audio-visual composition [4]. VisualEDU tests whether models can produce coherent educational problem-solving video and shows that increased task complexity exposes failures in coding, drawing, and feedback [3].

B. Speech synchronization and diagrams

Prasad and Mahapatra study timestamp-conditioned, structured whiteboard drawing synchronized with spoken narration [5]. EduIllustrate evaluates text-diagram educational explanations across subjects and reports that sequential visual anchoring improves consistency at lower cost than more elaborate workflows [6]. These results caution against a novelty claim that *all* narration-synchronized or low-cost diagram systems are absent from prior work. Explainora's narrower contribution is a practical combination of measured

voice-duration feedback, typed vector animation, locally repaired scene plans, and audited token expenditure in a complete video pipeline.

C. Closed-loop generation

CoAgent frames multi-shot video synthesis as plan-synthesize-verify with selective regeneration [7]. VISTA iteratively critiques visual, audio, and contextual fidelity and rewrites generation prompts [8]. These are genuine closed-loop predecessors. Explainora differs in the feedback signal and repair substrate: measured TTS seconds and executable vector-layout checks can be acted on locally or by narrowly targeted scene repairs. We therefore do **not** claim to introduce closed-loop video generation itself. Our hypothesis is that inexpensive, deterministic feedback can close several loops without paying for full video or full-script regeneration on every failure.

D. Comparison with published educational-generation systems

The studies below address related but non-identical outputs. **A number in another paper is not a matched Explainora baseline** unless the same prompts, duration, models or pricing basis, and acceptability test are applied. Reported success, model-judged scores, and human ratings use different definitions and must not share a ranking column.

TheoremExplainAgent's reported 93.8% and 0.77 are its own automated benchmark outcomes, not Explainora's human-acceptable criterion. Manimator's evaluation tables, PresentAgent's five-document table, EduIllustrate's static-diagram study, and VisualEDU's 200-task benchmark support the remaining rows. The absence of token disclosure in several studies is a measurement boundary, not evidence of zero token use.

III. IMPLEMENTED SYSTEM DESIGN

Figure 1 depicts the implemented production batch path.

A. Request admission and durable execution

The browser submits a prompt, requested seconds, audience, style, and optional upload to the video-creation API. The API authenticates the Supabase session, checks AI-provider readiness, consumes a generation rate limit, checks plan quota and duration, validates prompt and upload size/type, and reserves a durable job. It returns a job ID and polling URL with HTTP 202. The web process does **not** render the video in a request thread. A separate worker atomically claims a queued job through a Supabase database function, maintains a heartbeat, observes cancellation, and runs the generation pipeline. The production configuration disables incremental playback, selects a Piper ONNX voice, and bounds concurrent visual planning. Queue, web, worker, and storage are operational architecture, not novel LLM algorithms.

An uploaded document is extracted and saved privately; the worker downloads and re-extracts its text. However, the production batch handoff currently omits that extracted text from both the director and visual-planner inputs. This is a traceable functional limitation. Research evaluation of the present implementation must use prompt-only tasks, or document grounding must be repaired and separately verified before document tasks enter the paper.

B. Director, preflight, and the within-job planning loop

The director derives a target word count and scene count from the duration budget, allocates word and semantic-beat contracts to each scene, and calls the configured director model. Long-form requests can be split into hierarchical strategy and scene batches. The returned plan is normalized and compiled into narration beats, visual prompts, semantic contracts, and continuity instructions. **Before any audio is synthesized**, a deterministic duration preflight checks total and per-scene spoken-word contracts. If deficient, bounded repairs (configured to at most three attempts) retain accepted scene content. The director also checks unsupported empirical quantities against the user prompt and any passed source; it first applies deterministic qualifying changes where possible, then may invoke a verifier and targeted factuality repair. This is a real **generate -> validate -> selective repair -> validate** loop, but its duration feedback is a *word-count proxy*, not measured audio from the current job.

C. Cross-job speech-duration feedback and current-job timing

Let (w_i) be the word count of raw TTS scene (i), and (y_i) its measured raw-audio duration under voice profile (v). The profile ID hashes the TTS backend, model filename, voice, speed, and language. Once at least `MIN_CALIBRATION_SAMPLES` valid observations exist, `duration_calibrator.model()` uses a bounded median seconds-per-word estimate:

$$a_v = clip[median_i(y_i / w_i), a_{min}, a_{max}];\ W_{j+1} = round[D_{j+1}(1-r_v) / a_v]$$

where (r_v) reserves a configured fraction for pauses. The next job's director receives (W_{j+1}) as its narration budget. `record_text_sample` accepts only unmodified raw-TTS measurements with plausible speaking rates; it rejects tempo-modified or padded audio. Other text features are recorded and used by a separate duration estimator, but the **current word-budget inversion uses the median seconds-per-word model**, not a multifeature least-squares regression. Cold start falls back to a fixed default rate. This is the implemented *cross-job* loop: **earlier plan -> raw Piper speech -> measured seconds -> profile sample -> later job's word budget**.

For the current job, `_fit_narration_duration` synthesizes each scene **once**, measures it, allocates final scene durations, and adds boundary silence to short scenes. It neither re-prompts the director after hearing the speech nor resynthesizes the current script. Overshooting raw speech can remain overshooting; padding is not an exact-duration guarantee. The evaluation must separately report raw-speech error, final-MP4 error, pause seconds, and single-pass rate. Claims of "audio-driven in-job script correction" or "100% exact target duration" are unsupported by this path.

D. Visual scenes, local acceptance, and encoding

The batch worker asks `create_visual_plan` for each scene's structured elements using its visual prompt, semantic contract, continuity instruction, and beat timeline. Plans can be generated concurrently, although the Oracle Compose file currently caps `MAX_VISUAL_WORKERS=1`. `ContinuityCoordinator.submit` commits plans in scene order and reconciles recurring objects. Subsequent local functions materialize final visual state, repair post-continuity text/layout, normalize the plan, apply graphic style, assess semantic anchor coverage, and run a pre-render quality gate. If that gate fails, a deterministic `semantic_fallback_plan` competes with the original in a candidate registry; the configured active gate chooses the better candidate. SVG rendering can isolate a faulty element or use a safe fallback, then writes SVG, JSON scene plan, and PNG to worker scratch space. This sequence is *local validation and candidate selection*, not a claim that every failure triggers a fresh LLM call.

The video stage parses optional typed animation directives; if too few exist, it creates beat-window animation directives locally. When the fast SVG animation renderer is available, it renders narrated animated segments. Otherwise it attempts cumulative static beat states and

TABLE I. Comparison with published educational-generation systems; tasks and scores are not matched.

System	Input and output	Published evaluation or resource evidence	Difference relevant to this study
Explainora (this work)	Open question -> 120 s narrated vector-scene MP4	Four local quality-dial files playable; median 49,600.5-55,949 reported LLM tokens; no blinded human score	Voice-profile timing feedback for later jobs; local scene acceptance and rendering; prompt-only batch evaluation only
TheoremExplainAgent [1]	STEM theorem -> Manim video, usually over 5 min	240-theorem benchmark; o3-mini automated success **93.8%**, overall score **0.77**; model-dependent Table 5 usage/cost below	Generates and repairs full animation programs; longer, theorem-specific task, so its costs are context rather than a matched comparator
Manimator [2]	Prompt or research paper -> Manim animation	TheoremExplainBench overall **0.845** in its reported automatic comparison and **0.738** in its human evaluation; per-video token totals not reported	Two-stage scene-description -> animation-program generation; scores and costs cannot be paired with Explainora's local prompts
PresentAgent [4]	Long document -> narrated slide-style MP4	On the paper's five-document result set, GPT-4o-Mini variant: quiz accuracy **0.64**, model-judged mean video score **4.67/5**; token totals not reported	Document-grounded slides, not animated vector scenes; its 1-5 score is VLM-generated, not blinded human rating
EduIllustrate [6]	K-12 problem -> interleaved text and *static* diagrams	230-problem benchmark; Kimi-K2.5 **80.8%** at **US$0.12/problem**; sequential anchoring improves visual consistency **13%** at **94% lower cost** in its own ablation	Important low-cost visual-explanation precedent, but it does **not** produce narrated video
VisualEDU [3]	200 problem-solving tasks and a video-generation evaluation framework	Nine tested VLMs; advanced models score below **56%** on difficult tasks in its reported setting; no comparable per-video token ledger	A benchmark and code/visual-feedback framework, not a service with directly comparable output price

ultimately a static PNG segment. FFmpeg encodes each scene with its audio and concatenates the final MP4. The manifest includes runtime configuration, duration history, scene metadata, and model-usage summary. Evaluation files are created locally. `supabase_store.complete_video` uploads MP4, manifest, evaluation, and scene images to private Storage and writes `videos`, `video_assets`, and related scene records. Only then is the durable job marked completed. This is **finished-file publication**; the separate incremental-streaming implementation is disabled in Oracle production.

E. Implementation-status audit

F. Token accounting and measurement boundary

Every LLM call is logged with model, stage, prompt tokens (P_i), completion tokens (C_i), cached prompt tokens, reasoning tokens when available, retries, latency, and estimated price. Total language-model token demand is

$$T_{job} = \sum_{i=1}^{m} (P_i + C_i)$$

The run ledger aggregates provider-reported usage for successful calls and responses whose usage fields are available. It records attempted failures, but a request that fails before a provider returns usage may have **unknown** token consumption; the paper must not report that unknown value as zero actual provider usage. Cached and reasoning tokens are reported separately to distinguish billing from raw token demand. Repair calls and successful retries are included. Fallbacks executed locally add no model calls but must still count as output-quality failures when unsuitable. A token-efficiency comparison must condition on usable outputs:

$$T_{successful} = \sum_{j=1}^{N} T_j / N_{accepted}; \quad R_T = 1 - T_{Explainora} / T_{baseline}$$

The quoted reduction (R_T) is meaningful only after output criteria, duration, provider token reporting, and test prompts are matched. Dollar cost must be reported independently because model prices and free-tier credits can make a token-heavy run appear cheap.

IV. EXPERIMENTAL METHODOLOGY

A. Reproducible prompt-only test set

For a submission-grade study, freeze a commit, container image, environment configuration **without secrets**, provider/model versions, model price date, voice-model hash, FFmpeg version, and hardware profile. Use a preregistered, held-out *prompt-only* set with an equal number of questions across physics, biology, computing, mathematics, and everyday science; choose the count from a power or precision calculation before seeing the results. Each item specifies learning objectives, audience level, style, and a target duration drawn from the actual supported plan tiers. Use repeated runs per prompt to expose stochastic planner failures. Uploaded-document tasks are excluded until the batch source-text gap in Section III-A is repaired and verified with an integration test. Split earlier jobs from evaluation jobs chronologically so the duration calibrator cannot learn from its own test output.

Run each item through the same production **batch** path: authenticated API admission, queued Supabase job, worker claim, director planning, Piper ONNX synthesis, structured visual generation, MP4 encoding, private Storage publication, and job completion. Observe job outcomes from both API state and durable records. A run is not successful merely because HTTP 202 returned a job ID; it must produce a playable MP4, a complete manifest, and an acceptable teaching/visual rating.

B. Metrics

Report (1) job completion and *human-acceptable* video fractions; (2) median, interquartile range, and 95th-percentile input/output/total tokens per attempt and per acceptable output; (3) successful-retry tokens and calls whose usage is unavailable; (4) price-estimated API cost, render CPU time, and end-to-end latency; (5) pre-padding raw-speech duration error and post-padding final-MP4 error separately; (6) raw TTS pass count, director preflight repair count, boundary-pause seconds, and semantic fallback frequency; (7) detected factual-quantity violations, visual legibility, scene-beat correspondence, cross-scene continuity, and animation/static fallback distribution; and (8) blinded human clarity and learning-usefulness

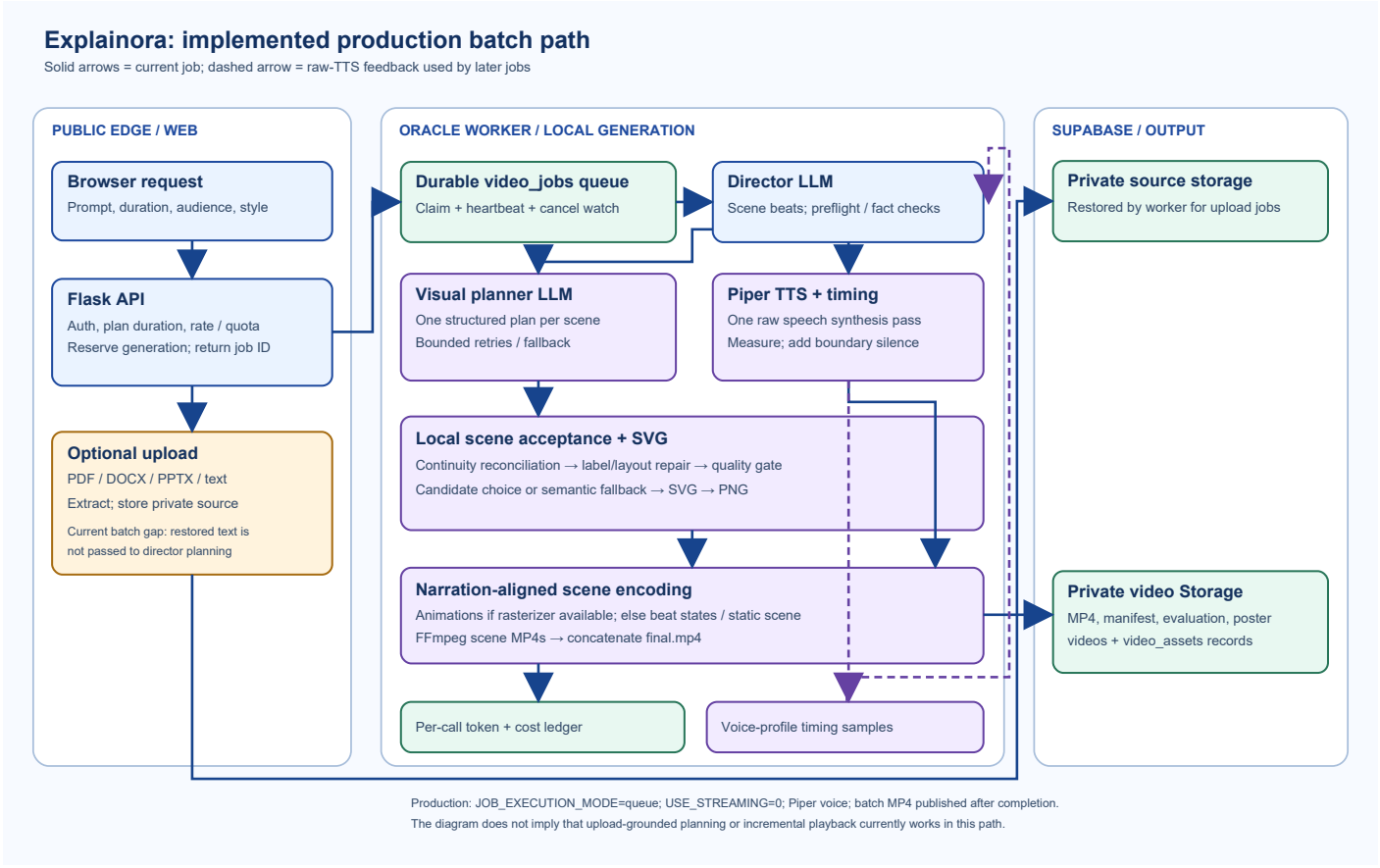


Fig. 1. Implemented production path. Solid lines represent current-job control and data flow; the dashed line is raw-TTS feedback used for later jobs. The figure explicitly marks the uploaded-document grounding gap. It does not depict incremental playback as a production feature.

scores. Source faithfulness is deferred until document grounding works. Use at least two raters per video, disclose expertise, report inter-rater agreement and bootstrap confidence intervals, and judge several unseen topics rather than one successful demo.

C. Token-cost comparison and market interpretation

TheoremExplainAgent reports approximately **350,000-1,120,000 input tokens plus 84,000-154,000 output tokens** and **US$0.10-US$4.67** per video across its tested model configurations, with roughly 1,120-2,380 s inference time [1, Table 5]. Those are published, study-specific figures, **not** a universal current-market average. Explainora's design makes a smaller token budget plausible, because SVG geometry, animation scheduling, cross-scene continuity, TTS, encoding, and many repairs occur locally; however, a reduction against code-generating video must be measured on matched tasks and must include failed calls. No external matched baseline has been implemented or run in this checkout. The following table gives **descriptive resource context only**; it does not normalize video length, task difficulty, quality, provider pricing date, or accepted-output rate.

The published configurations above come from TheoremExplainAgent's original cost table, which assumes four fixed animation programs and seven scenes per video. Its task, output length, evaluation criteria, model mix, and price date differ from Explainora's local pilot; no percentage reduction or superiority claim follows from these absolute figures. The actual controlled comparison below is **within Explainora**: fast planner routing versus the balanced planner, with identical prompts, audience, style, and 120 s targets on two topics. Its denominator is *completed playable MP4s*, **not human-acceptable outputs**.

The cost calculation uses provider-reported per-model input and output totals and model-list rates checked on 15 September 2026: Cerebras `gpt-oss-120b`, Groq `qwen/qwen3.8-27b`, and Groq `openai/gpt-oss-20b`. It covers every **reported** model/token total in these four completed jobs; it is not an invoice, excludes free-tier credits, hosting, tax, and unknown consumption if a request fails without usage. The project's older built-in ledger lacked prices for Qwen calls, which is why its stored subtotal must not be substituted for this estimate. The fast runs made 15 and 18 LLM calls versus 11 and 11 balanced calls, consistent with more retry/repair overhead, but this tiny sequential sample cannot isolate the causal source of the token increase.

D. Local implementation pilot (diagnostic, not benchmark)

To replace factual blanks with observed evidence, we ran the built frontend and local Flask backend, verified `/healthz` and `/readyz`, and submitted two 120-second beginner, clean-diagram, prompt-only topics with the configured Cerebras director, Groq visual planner, Piper voice, SVG rendering, and FFmpeg. Supabase settings were emptied in the **local process only** to prevent test videos or records from entering the shared production database. An initial tides attempt exposed a `TypeError` in connector counting during visual normalization. After a narrow fix and regression test, the same prompt completed. The three attempts therefore span different code snapshots and must not be pooled as a production reliability estimate.

Both completed files were inspectable as MP4s, but sampled scene images had sparse icons, generic placeholders, and/or overlapping

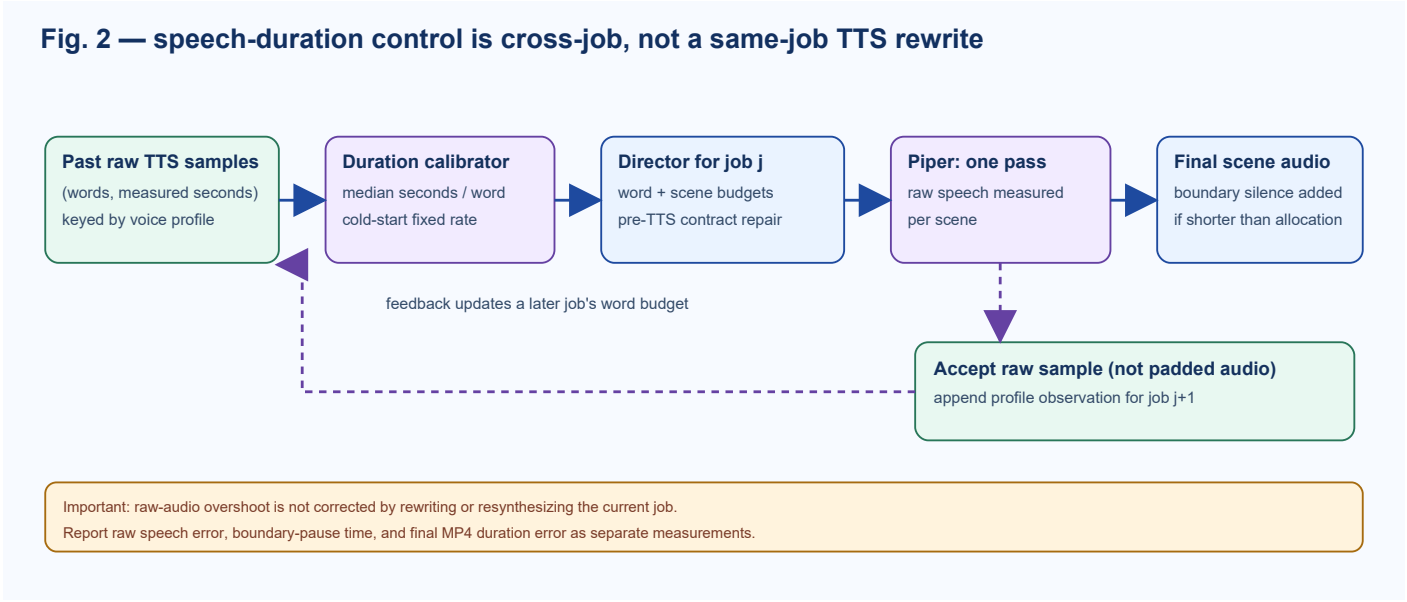


Fig. 2. Two different controls must not be conflated: word-contract repair occurs before TTS within job j; measured raw speech from job j updates the voice model for job j+1.

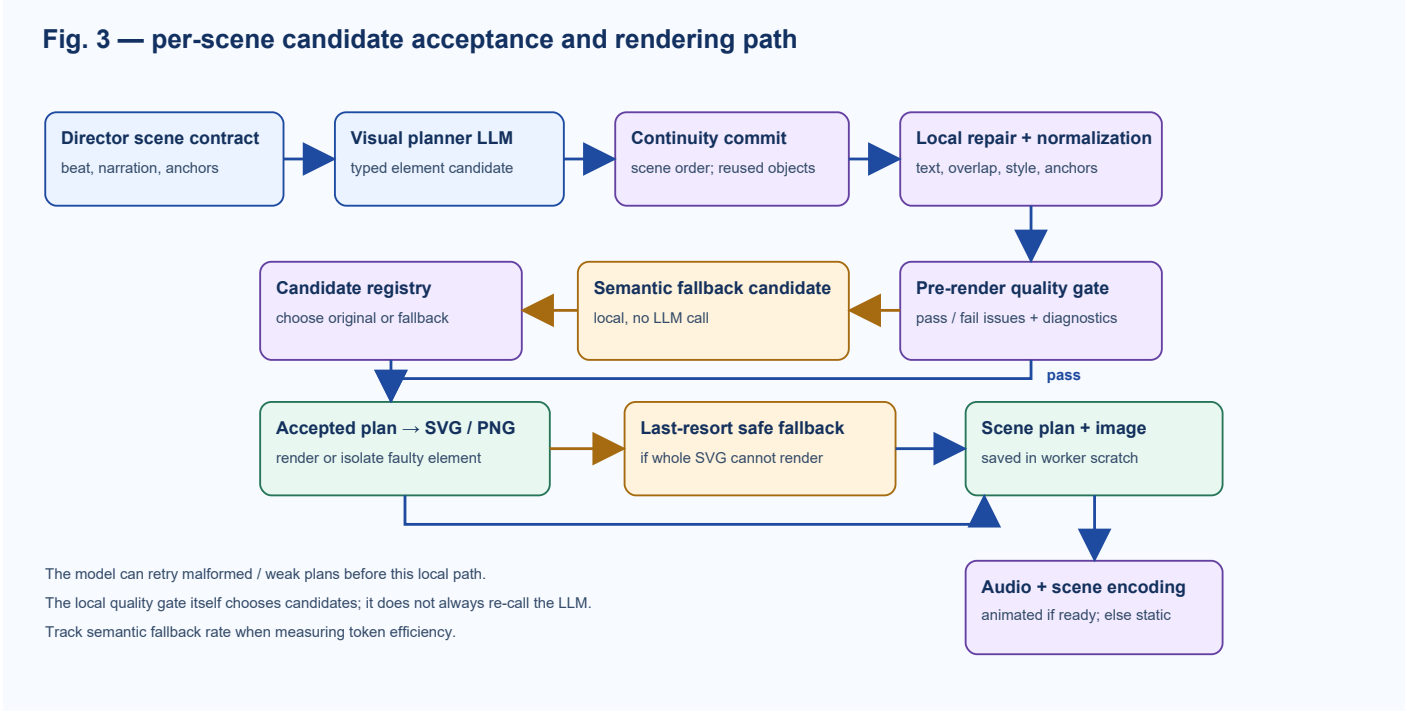


Fig. 3. The production scene path accepts, locally repairs, or selects a deterministic fallback. Only the planner's own retries and explicitly invoked repairs consume new model tokens.

labels. There is no blinded human clarity rating or accepted-video count. The GPS and tides prompts are different, and the tides rerun followed a code change, so neither the 71,936-to-52,704 difference nor published token figures from [1] can be called an experimentally measured saving. The subtotal cost excludes unpriced provider/model calls and must not be described as whole-video expenditure.

The newer matched quality-dial runs use one local implementation snapshot after that fix. Supabase was again disabled **only for these local evaluation processes**; the models, Piper, vector rendering, and FFmpeg ran, but the production authentication, durable queue, and private Storage publication did not. A sandboxed first GPS attempt failed at planning because local socket access was forbidden; an isolated network-enabled process then completed the four runs. That failed attempt has no provider-returned usage and is excluded from the completed-output medians, not counted as zero cost. The per-topic usage and price inputs are reported below; aggregate medians in Section IV-C follow directly from those rows. Execution scripts and sensitive raw call logs are not included in the paper package.

FFprobe confirmed H.264 video, AAC audio, and manifest-consistent durations in all four files. However, sampled GPS and tides frames showed generic icons, overlap, or truncated text, including a cut-off

TABLE II. Implementation status of the production-batch mechanisms.

Paper mechanism	Observed production-batch behavior	Accurate status
Durable queued generation	Authenticated admission reserves a job; a separate worker claims it atomically	Implemented in production configuration
Director scope, scene beats, word preflight	Word and beat contracts are compiled and checked before TTS	Implemented before TTS
Measured audio changes same job's script	Timing stage measures one TTS pass, then can add boundary silence	**Not implemented**; no audio-driven script revision
Measured audio informs later jobs	Valid raw-speech samples update the profile used for later word budgets	Implemented, conditional on enabled calibration and valid samples
Structured SVG visual plan and local repair	Ordered continuity, layout repair, semantic gate, and fallback selection	Implemented; planner failure may still produce simplistic fallback
Beat-timed animation on every scene	Animated segments are attempted; static states or PNG segments remain possible	Conditional; static fallbacks exist
Uploaded-document grounding in production batch	Extracted text is restored but not passed to batch director or visual planner	**Not implemented end to end**
Incremental playback during render	Streaming path exists separately but production runs finished-file batch encoding	**Disabled** in production
Completed video in private Storage	Publication precedes durable completed status	Implemented and required before completed status

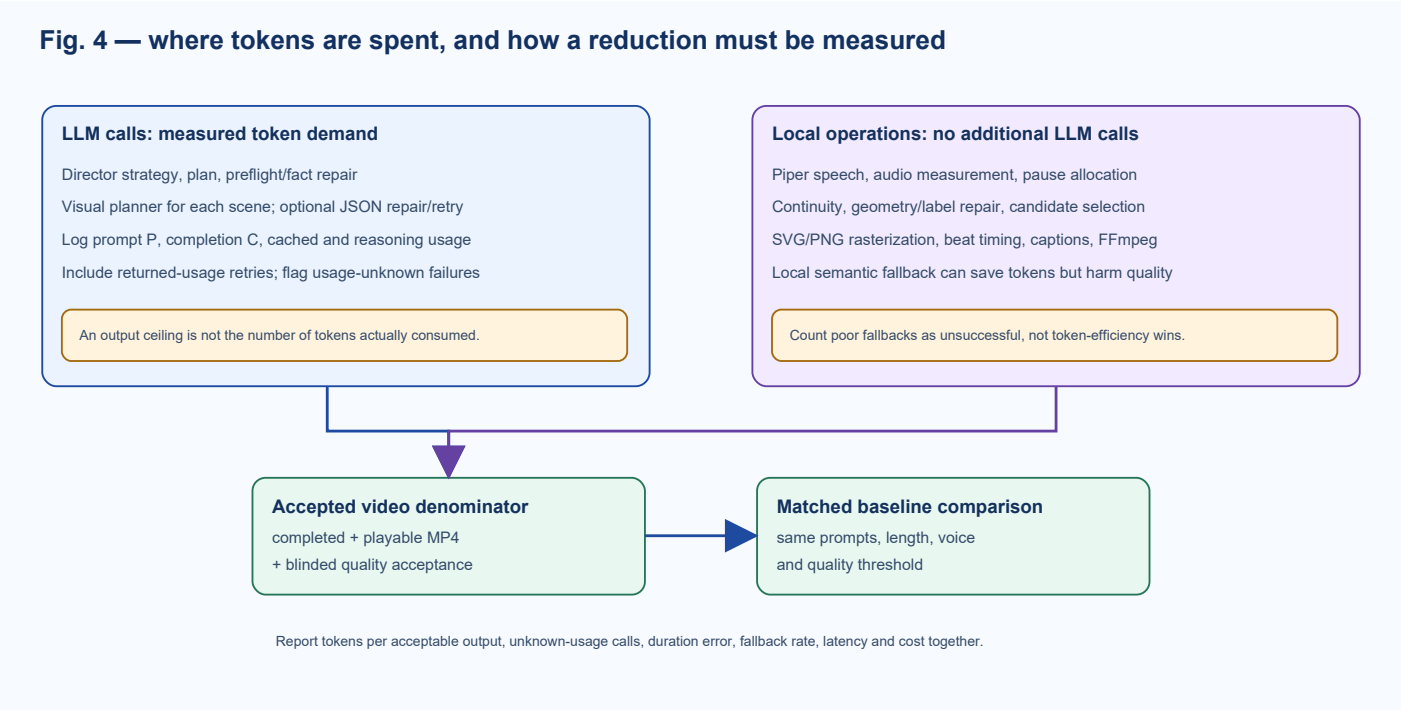


Fig. 4. Token savings are a testable systems hypothesis: report all provider-observed usage and divide by the number of playable, quality-accepted outputs. Local fallbacks are not successes by definition.

final tides headline in the fast run. Passing an automated scene gate or producing an MP4 is therefore not evidence of human instructional acceptability. These runs are a small *paired diagnostic*, not an external benchmark; they were sequential, the voice profile could update between attempts, and no blinded raters or repeated random seeds were used.

E. Ablations and baselines tied to the implementation

Voice calibration. Compare the production setting `ENABLE_WPS_CALIBRATION=1` with the existing fixed-rate mode `ENABLE_WPS_CALIBRATION=0`, holding Piper voice and prompts fixed. Evaluate cold-start and warm-profile phases separately. The expected effect is reduced *raw-speech* duration error on later jobs, **not** fewer post-TTS revisions, because the batch path never performs such revisions.

Visual acceptance. Compare the active pre-render gate with its configured shadow mode (`VISUAL_QUALITY_SHADOW_MODE=true`). Report selected fallback fraction and rated clarity as well as tokens; local fallback may save calls yet worsen pedagogical quality. The quality gate's existing shadow switch provides a direct ablation without altering the renderer.

Animation. Compare `ANIM_ENABLED` on/off on the same accepted scene plans, and separately report the fraction of scenes that actually rendered animation versus cumulative/static fallback. Disabling animation may change visual teaching quality but should not be described as eliminating a separate animation LLM call unless logs show such a call existed.

Continuity and full-regeneration comparison. A continuity-disabled ablation and a Manim/full-regeneration baseline need explicit experimental implementations and tests; they are

TABLE III. Reported language-model usage in non-equivalent educational-video studies.

System / configuration	Video task	Input tokens	Output tokens	Total tokens	Estimated API cost per video
Explainora, balanced local pilot	Two open-domain topics, 120 s target	27,432 median	22,168.5 median	49,600.5 median	US$0.07646 median, September 2026 list rates
Explainora, fast local pilot	Same two topics and target	39,052.5 median	16,896.5 median	55,949 median	US$0.01557 median, September 2026 list rates
TheoremExplainAgent, GPT-4o [1, Table 5]	Theorem video, typically over 5 min	350,000	84,000	434,000	US$1.71, study rates
TheoremExplainAgent, Gemini 2.0 Flash [1, Table 5]	Same theorem-video study	595,000	119,000	714,000	US$0.10, study rates
TheoremExplainAgent, o3-mini [1, Table 5]	Same theorem-video study	434,000	154,000	588,000	US$1.16, study rates

TABLE IV. Within-system, two-topic exploratory quality-setting pilot.

Metric, two-topic exploratory pilot	Fast Explainora	Balanced Explainora (within-system reference)	Fast minus balanced
Technically playable videos	2/2	2/2	0 percentage points
Human-acceptable videos	Not measured	Not measured	Not estimable
Median prompt tokens per playable video	39,052.5	27,432	**+42.4%**
Median output tokens per playable video	16,896.5	22,168.5	**-23.8%**
Median total tokens per playable video	55,949	49,600.5	**+12.8%**
Median model-list-price estimate per playable video	US$0.01557	US$0.07646	**-79.6%**
Median absolute final-duration error	2.60 s	2.04 s	**+0.56 s**
Blinded human clarity score	No raters recruited	No raters recruited	Not estimable

proposed, not existing production modes. Their matched outputs must be judged at the same educational quality threshold before quoting token reductions. TheoremExplainAgent's published Table 5 remains contextual, because it uses longer theorem-specific videos and different models [1].

V. DISCUSSION AND LIMITATIONS

Symbolic vector graphics provide sharp labels, reproducible geometry, and inexpensive local animation, but cannot reproduce the photorealism or semantic richness of diffusion-based footage. Model-generated scene plans can still produce sparse icons and arrows when a planner call fails or a fallback is overused; therefore, fallback rate and visual-density ratings must be included in the results. Provider quotas, unsupported models, invalid JSON, and network failures can create apparent token savings by producing incomplete or low-quality videos. Closed-loop duration calibration also depends on a stable TTS profile and enough raw observations; cross-language and cross-voice generalization is not assumed. Source-grounded factuality checks are strongest for traceable quantities and do not replace expert review.

The system's internal usage logs do not reveal commercial competitors' private model usage. Therefore, the manuscript should not claim that Explainora consumes fewer tokens than "all products presently on the market." It can claim a reduction against a **specified, reproduced baseline** if the evaluation supports it, and discuss published token ranges from [1] as context. Price estimates reflect list prices and exclude credits, taxes, hosting, and human correction. Privacy and licensing of uploaded documents must be addressed in the final study protocol.

VI. CONCLUSION

Explainora demonstrates a design in which LLMs decide instructional structure and semantic scene content, while local software handles timing measurement, layout, animation, continuity, rendering, and much of the repair. Its verified speech-timing loop updates **later** jobs, while within-job planning and visual checks selectively reject or correct deficient candidates before publication. In the newer two-topic, four-file local quality-dial pilot, fast planner routing cut the median *estimated list-price model cost* by 79.6% relative to balanced routing but **increased** median total LLM tokens by 12.8%; median absolute final-duration error rose from 2.04 to 2.60 s. All four attempts produced playable MP4s, yet sampled visual defects and the absence of blinded human ratings prevent treating them as accepted educational outputs. Earlier pilot attempts also exposed a visual-normalization failure, illustrating that failed generations can consume substantial tokens. A frozen, replicated, external matched-task evaluation of **tokens and cost per successful, human-acceptable video** remains necessary before a market-comparison or token-saving claim can be made.

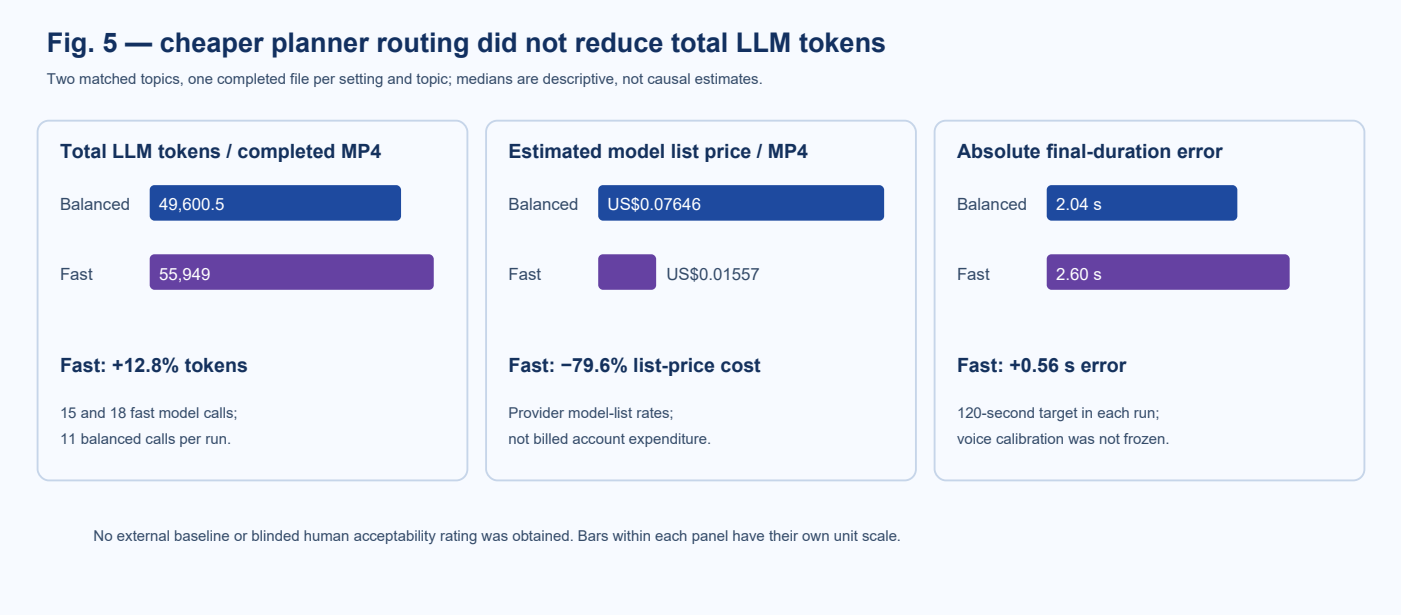


Fig. 5. Descriptive medians for two matched prompt-only topics. Fast routing is cheaper at published model-list rates but spent more reported LLM tokens and had larger final-duration error. Neither setting was accepted by blinded human raters, and this is not an external comparison.

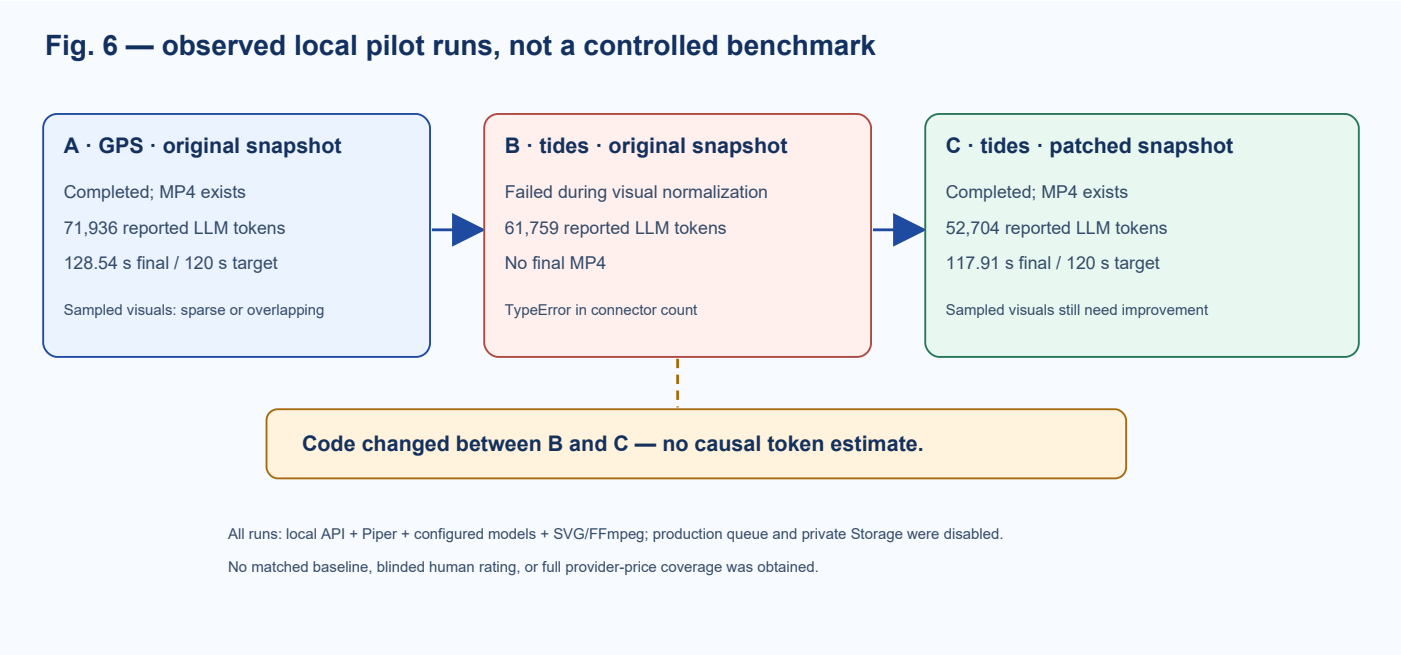


Fig. 6. Actual local pilot outcome sequence. The connector-count patch separates the first two runs from the third. Production queue, authentication, and private Storage publication were deliberately disabled, so this does not validate the full production path.

TABLE V. Local implementation attempts across a visual-normalization fix.

Local attempt	Observed outcome	Reported prompt / completion / total LLM tokens	Raw speech / final MP4 vs. 120 s	Price coverage
GPS, `1e9480896b60`, original local snapshot	Completed; six scenes and MP4	34,163 / 37,773 / **71,936**	128.57 / 128.54 s; final error +8.54 s	US$0.01107 **known-priced subtotal**; 10 unpriced calls
Tides, `71af28dcac91`, original local snapshot	Failed at visual normalization; no MP4	33,887 / 27,872 / **61,759**	No final duration	US$0.01047 **known-priced subtotal**; 11 unpriced calls
Tides, `67af6c3c99ea`, patched local snapshot	Completed; six scenes and MP4	25,638 / 27,066 / **52,704**	112.52 / 117.91 s; 5.40 s boundary pause; final error -2.09 s	US$0.00847 **known-priced subtotal**; 7 unpriced calls

TABLE VI. Per-topic outcomes of the paired local quality-setting pilot.

Topic	Setting / local job ID	Reported input / output / total tokens	Final duration	Estimated model-list-price cost	Pre-render gate failures
GPS	balanced / `78eb2b313b9d`	27,286 / 22,789 / 50,075	120.89 s	US$0.07788	1/6 scenes
GPS	fast / `809e0262ef31`	36,018 / 16,047 / 52,065	118.94 s	US$0.01421	1/6 scenes
Tides	balanced / `52f76a38d972`	27,578 / 21,548 / 49,126	116.80 s	US$0.07505	0/6 scenes
Tides	fast / `98a9e6c46b74`	42,087 / 17,746 / 59,833	115.85 s	US$0.01694	1/6 scenes

REFERENCES

[1] M. Ku, C. H. Chong, J. Leung, K. Shah, A. Yu, and W. Chen, "TheoremExplainAgent: Towards video-based multimodal explanations for LLM theorem understanding," in *Proc. 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 6663-6684, doi: 10.18653/v1/2025.acl-long.332. [Online]. Available: https://aclanthology.org/2025.acl-long.332/

[2] S. P, V. Jain, S. Golugula, and M. Sai Sathvik, "Manimator: Transforming research papers into visual explanations," *arXiv preprint* arXiv:2507.14306, 2025. [Online]. Available: https://arxiv.org/abs/2507.14306

[3] H. Chen, T. Shi, P. Huang, Z. Li, J. Pan, Q. Chen, and L. He, "VisualEDU: A benchmark for assessing coding and visual comprehension through educational problem-solving video generation," in *Findings of the Association for Computational Linguistics: EMNLP*, 2025, pp. 16363-16394, doi: 10.18653/v1/2025.findings-emnlp.889. [Online]. Available: https://aclanthology.org/2025.findings-emnlp.889/

[4] J. Shi, Z. Zhang, B. Wu, Y. Liang, M. Fang, L. Chen, and Y. Zhao, "PresentAgent: Multimodal agent for presentation video generation," in *Proc. EMNLP System Demonstrations*, 2025, pp. 760-773, doi: 10.18653/v1/2025.emnlp-demos.58. [Online]. Available: https://aclanthology.org/2025.emnlp-demos.58/

[5] S. Prasad and P. Mahapatra, "Speech-synchronized whiteboard generation via VLM-driven structured drawing representations," *arXiv preprint* arXiv:2603.25870, 2026. [Online]. Available: https://arxiv.org/abs/2603.25870

[6] S. Bi, M. Zhang, Z. Li, X. Wang, K. Li, and A. Zhou, "EduIllustrate: Towards scalable automated generation of multimodal educational content," *arXiv preprint* arXiv:2604.05005, 2026. [Online]. Available: https://arxiv.org/abs/2604.05005

[7] Q. Zeng, K. Cai, R. Chen, Q. Lv, and K. Wang, "CoAgent: Collaborative planning and consistency agent for coherent video generation," *arXiv preprint* arXiv:2512.22536, 2025. [Online]. Available: https://arxiv.org/abs/2512.22536

[8] X. L. Do, X. Wan, H. Nakhost, C.-Y. Lee, T. Pfister, and S. Arik, "VISTA: A test-time self-improving video generation agent," *arXiv preprint* arXiv:2510.15831, 2025; listed by Google Research as forthcoming at IEEE/CVF CVPR 2026. [Online]. Available: https://arxiv.org/abs/2510.15831

SUBMISSION-CRITICAL EVIDENCE STILL NEEDED

1. Freeze a paper version of the code, model configuration, pricing date, and evaluation prompts. The repository's existing uncommitted changes are not a reproducible research release.

2. Export complete job manifests and usage traces with secrets and personal data removed. An older project note quotes results for 74 jobs, but those manifests are not presently available in this checkout; do not transfer the numbers into the manuscript unverified.

3. Run matched baselines and ablations. Treat successful output and blinded instructional quality as prerequisites for any token-saving headline.

4. Recheck reference metadata and publication status immediately before submission, especially 2026 preprints and the forthcoming VISTA venue.

5. Typeset with the exact IEEE conference or journal template after its venue and page limit are chosen.